

L AFC-Evict: A Large-Scale Counterfactual Benchmark for Learned Cache Eviction: [Experiments & Analysis]

Anonymous Author(s)

ABSTRACT

Learned cache-eviction research has outgrown small ad hoc evaluations, yet empirical comparison remains difficult because prior studies often rely on private traces, release-incomplete preprocessing pipelines, or supervision targets that are not comparable across papers. This paper presents L AFC-Evict, a benchmark package for learned cache eviction in caching systems built around decision-aligned counterfactual supervision. Each canonical candidate row represents one candidate victim considered at one full-cache miss decision and includes release-facing provenance fields, cache-state features, and a finite-horizon counterfactual label. The canonical loss label, y_loss , is the continuation-policy miss cost over a finite horizon after forcing that candidate eviction. The corresponding value label, y_value , is defined as $-y_loss$. The benchmark also exposes decision-level and pairwise benchmark views to support value prediction, best-candidate selection, pairwise preference learning, and label-surface characterization.

The evaluated open-release snapshot covers five selected trace families—cloudphysics, metacdn, metakv, twemcache, and wiki2018—and its release metadata record 277,995,072 candidate rows, 2,363,286 decision-view rows, and a capped pairwise sample of 1,000,000 rows. Two additional families, brightkite and citibike, remain excluded from the open subset. Beyond benchmark definition, release schema, validation, and reproducibility, the paper reports full candidate-label summaries, a linear y_loss regression baseline, a best-candidate selector induced by that linear score, and pairwise-sample-backed feature baselines. Across all candidate rows, y_loss has mean 7.7561, standard deviation 4.6542, and 5/25/50/75/95th percentiles of 2/4/7/11/16; the mirrored y_value summary confirms the sign-flipped construction. The linear regression baseline is intentionally weak (test MAE 3.8739, RMSE 4.7152, $R^2 = 0.0026$), while the derived best-candidate selector reaches 0.8753 optimal-selection rate with 0.1251 mean regret. The shipped pairwise sample’s A/B orientation is assigned by a deterministic hash over stable decision and pair-identity fields rather than by candidate-identifier ordering, giving a balanced non-tie split (about 49.8%/50.2%); on the resulting non-tie subset (121,738 of 1,000,000 rows), a logistic feature-difference baseline reaches 0.9660 accuracy with 0.0824 log loss after joining released candidate features back to the capped sample for evaluation only. These results demonstrate benchmark executability rather than online policy quality, and they are not evidence of online-policy superiority. The labels are therefore interpreted as decision-aligned finite-horizon counterfactual targets rather than offline-optimal supervision.

CCS CONCEPTS

• **Information systems** → **Data management systems**; • **Computing methodologies** → *Machine learning*.

KEYWORDS

learned cache eviction, benchmark datasets, counterfactual supervision, cache replacement

1 INTRODUCTION

This paper presents L AFC-Evict, a benchmark for learned cache eviction built from real-trace-derived cache decisions and computed finite-horizon counterfactual labels. The contribution is not a new eviction policy. It is a reusable benchmark surface for caching research that exposes candidate-level supervision, decision-level summaries, a shipped pairwise sample, and release-oriented metadata and validation. The motivating gap is now familiar in learned systems work: policy papers often compare methods under different traces, different preprocessing conventions, different supervision targets, and different release practices, which makes cross-paper empirical claims harder to interpret than the headline numbers suggest.

L AFC-Evict is designed to address that gap by making the supervised unit of comparison explicit. Each canonical candidate row corresponds to one candidate victim at one full-cache eviction decision, with decision metadata, candidate features, and a continuation-policy-specific finite-horizon counterfactual label. The release stores this label as y_loss , defines y_value as $-y_loss$, and treats both as properties of a candidate within a decision rather than as properties of a fully optimized future policy. From the same underlying release, the benchmark also exposes a decision view and a pairwise sample so that value prediction, within-decision best-candidate selection, pairwise preference prediction, and label-distribution characterization can be studied under a shared artifact. In that sense, L AFC-Evict is not merely a raw trace repository either: it sits above the trace layer and packages a decision-aligned supervision layer for benchmark reuse.

The evaluated open-release snapshot is already large enough to make that benchmark framing substantive. Table 1 reports manifest-backed release scale of 277,995,072 candidate rows, 2,363,286 decision-view rows, and a shipped pairwise sample of 1,000,000 rows across five selected trace families. Table 8 summarizes the decision view by family, capacity, horizon, and split. Table 9 shows that decision complexity is nontrivial even before any model training: in the decision view, candidate counts range from 32 to 256, with median 64 and mean 117.63. Table 5 reports coverage of 118,635 unique decisions in the pairwise sample. These numbers scope the benchmark, and the evaluated release package passed end-to-end validation. The paper complements those release descriptors with row-level y_loss/y_value summaries and lightweight baselines for value regression, best-candidate selection, and pairwise preference prediction.

The release also makes a deliberate boundary between raw data provenance and benchmark packaging. L AFC-Evict does not claim

authorship of the underlying traces, and the open subset is intentionally narrower than the broader generated corpus. Instead, the benchmark contribution is the releaseable supervision artifact derived from those traces. It includes the schema, the task views, the manifest, the checksums, and the validation machinery needed for reuse. That separation matters scientifically. The labels are finite-horizon and continuation-policy-specific decision targets rather than offline-optimal targets. It also matters operationally because redistribution constraints affect what can responsibly appear in the open subset.

To our knowledge, existing public cache-replacement artifacts do not provide a reusable candidate-level counterfactual supervision table at this scale. The novelty claim is therefore about the released supervision artifact, schema, validation surface, and benchmark tasks rather than about a new eviction policy.

The paper therefore makes a benchmark claim rather than a deployment claim. First, learned cache eviction can be studied through decision-aligned candidate supervision instead of only through end-policy comparisons. Second, one release can support multiple related tasks through candidate, decision, and pairwise views. Third, release engineering and validation are part of the scientific contribution because they determine whether later work can reproduce or extend the benchmark. Fourth, the paper focuses on benchmark construction, release structure, characterization, and simple baselines without overstating downstream systems impact or claiming model coverage beyond the evidence reported here. The baselines are deliberately modest in that sense: linear value regression remains weak out of sample, the induced best-candidate selector is useful as a reproducible benchmark floor rather than as evidence of production-quality policy behavior, and the new pairwise results show decision-relevant signal in released candidate features without converting the paper into a full-policy study.

The rest of the paper proceeds as follows. Section 2 reviews the learned cache-eviction setting and motivates decision-aligned supervision. Section 3 defines the benchmark scope and release-governance boundary. Sections 4 and 5 describe the label definition, schema, and derived views. Section 6 explains release construction and validation. Sections 7 and 8 define the benchmark tasks and the current characterization surface. Section 9 covers limitations, ethics, and reuse constraints, followed by related work and conclusion.

2 BACKGROUND AND MOTIVATION: LEARNED CACHE EVICTION

At each full-cache miss, a caching system must evict one resident object before admitting the requested one. Classical policies such as LRU operationalize this choice through simple recency structure, while learned or learning-augmented approaches attempt to predict which eviction will lead to lower future miss cost. In practice, these approaches differ in feature sets, prediction targets, and evaluation methodology.

The main difficulty for a benchmark is that victim selection is structured, not i.i.d. A model does not act on one object in isolation; it acts within a decision set whose size, feature spread, and ambiguity vary across traces and capacities. A useful benchmark

therefore preserves decision boundaries instead of flattening them away.

This observation motivates the core LAFC-Evict design choice: represent supervision at the candidate-within-decision level. Each row corresponds to one candidate victim at one full-cache eviction decision, and the label records the finite-horizon counterfactual miss cost of forcing that eviction under a fixed continuation policy. This representation allows both pointwise learning and within-decision evaluation, while keeping the supervised object aligned with the actual cache-eviction choice.

From that candidate-level foundation, the benchmark derives two secondary evaluation surfaces. Grouping all rows with the same `decision_id` produces a decision view that records candidate count, optimal-set membership, tie structure, and regret summaries. Sampling candidate pairs from the same decision produces the shipped pairwise view, which turns within-decision comparisons into a simpler classification problem without discarding the original decision boundary.

A single full-cache miss can therefore appear in three complementary forms. If one decision exposes k admissible victims, the release records k candidate rows, one decision-group summary, and up to $k(k-1)/2$ within-decision pairwise comparisons before any sampling cap is applied. The manuscript keeps that task-view progression textual rather than introducing a second overview figure beside the main pipeline diagram.

3 BENCHMARK DESIGN

LAFC-Evict is organized around five layers: external raw traces, processed traces, generated features, generated labels, and benchmark tasks. The release package begins after candidate-row generation and focuses on converting existing generated data into a reusable benchmark package for cache-eviction benchmarking, with schema, validation, and release metadata.

The evaluated open subset is `lafc-evict-v0.1-open`. The evaluated release metadata identifies five selected families: `cloudphysics`, `metacdn`, `metakv`, `twemcache`, and `wiki2018`. The family registry also lists `brightkite` and `citibike` as excluded from the evaluated open subset for redistribution reasons. This scope boundary is a release-governance decision, not a statement that the broader generated dataset mix is automatically redistributable. Not every selected family appears in every split in the evaluated release; for example, `metacdn` contributes train and validation decision rows but no test decision rows. Split assignment is produced upstream of this release package, which only normalizes and validates split values rather than assigning them, so uneven per-family split coverage reflects that upstream construction rather than a benchmark design guarantee of uniform cross-family generalization.

The canonical benchmark unit is the candidate row. A single row corresponds to one candidate victim at one full-cache miss decision, so one eviction decision with $|C_t|$ admissible victim choices contributes $|C_t|$ candidate rows. Decision-level and pairwise benchmark objects are derived from that candidate-level base rather than collected independently. This design keeps the benchmark centered on cache-eviction decisions instead of conflating the benchmark contribution with end-to-end deployment claims.

The benchmark is designed to satisfy four requirements:

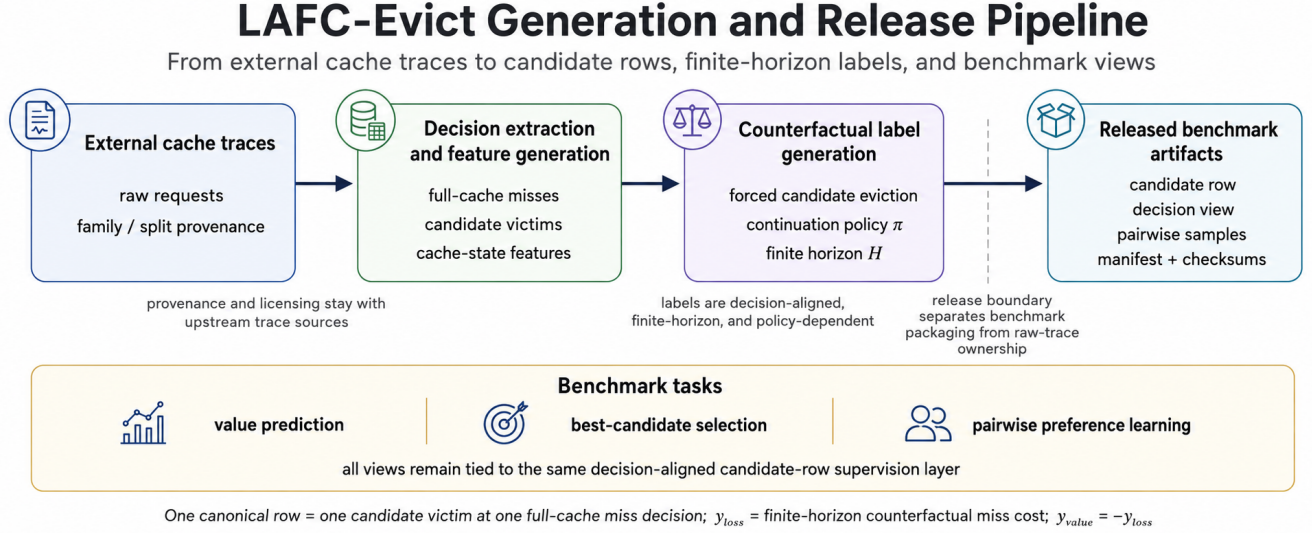


Figure 1: LAFC-Evict generation and release pipeline. The benchmark converts external cache traces into decision-level candidate rows, finite-horizon counterfactual labels, and released benchmark views for value prediction, best-candidate selection, and pairwise preference learning.

Table 1: Dataset scale from preserved-release metadata. Counts are manifest-backed or view-backed as noted.

metric	value
candidate rows	277,995,072
decision rows	2,363,286
pairwise-sample rows	1,000,000
preserved release size	2.7G
candidate parquet files	168
total release files	176
manifest file_inventory count	176
checksum entry count	175
selected trace families	cloudphysics, metacdn, metakv, twemcache, wiki2018
excluded trace families	brightkite, citibike
full validation status	passed

Table 2: Split counts from manifest-backed candidate-row metadata. These are candidate-row counts, not decision-view counts.

split	row_count
test	28,205,952
train	205,726,368
val	44,062,752

- (1) **Decision alignment.** Preserve within-decision structure and candidate competition.
- (2) **Task multiplicity.** Support pointwise, decision-level, and pairwise learning tasks from one release.
- (3) **Releaseability.** Separate public benchmark artifacts from upstream raw-trace ownership and hosting constraints.
- (4) **Validation.** Expose checks for schema, consistency, family scope, and checksums.

4 COUNTERFACTUAL LABEL GENERATION

The benchmark label is defined on candidate victims rather than on completed policies. Let $r_1, r_2, \dots$ denote a request sequence, and let t index a request r_t that misses when the cache is full. Write S_t^- for the cache state immediately before the eviction choice; the candidate set $C_t = S_t^-$ is every resident object, so $|C_t|$ equals the cache capacity (32, 64, 128, or 256 objects in the evaluated release), counted

in objects rather than bytes, with upstream object sizes not separately modeled. For any candidate $c \in C_t$, consider the intervention that evicts c , admits the missed request into the freed slot, and then follows a fixed continuation policy π for the next H requests, resolving every subsequent miss according to π ; in the evaluated release, π is LRU continuation rather than a Belady/offline-optimal rule. The resulting finite-horizon counterfactual loss counts those subsequent misses, so every miss contributes unit cost:

$$L_{t,H}^\pi(c) = \#\{\tau \in \{t+1, \dots, t+H\} : r_\tau \text{ misses in the induced trajectory}\}.$$

The release stores this quantity as y_{loss} , and it stores the corresponding value label as $y_{value} = -y_{loss}$. The triggering miss at time t is common to all candidates and is therefore not the decision-discriminating part of the label.

This label has two consequences for the paper. First, it creates a common numerical target for regression-style benchmark tasks. Second, it induces structured decision-level objects. For a fixed decision (t, S_t^-, C_t) , the benchmark-optimal candidate set is

$$C_t^* = \arg \min_{c \in C_t} L_{t,H}^\pi(c),$$

Table 3: Compact summary of candidate-row schema groups and derived-view fields.

Column group	Notes
Decision metadata	family, capacity, horizon, split, IDs
Candidate identity	candidate page ID
Labels	y_loss, y_value
Feature groups	request, candidate, cache, disagreement, rate
Derived decision fields	tie and regret summaries
Derived pairwise fields	A/B labels and regret differences

and the per-candidate regret is

$$\Delta_{t,H}^{\pi}(c) = L_{t,H}^{\pi}(c) - \min_{c' \in C_t} L_{t,H}^{\pi}(c').$$

These quantities define best-candidate selection targets, pairwise preferences, and tie structure within a decision.

This supervision is a generated counterfactual label family, not an upstream annotation. It is finite-horizon, not asymptotic. It is tied to a continuation policy, not a universally optimal control objective. The label is therefore decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label in the sense of solving the full future control problem. These boundaries are central to the scientific positioning of the paper.

5 DATASET SCHEMA AND VIEWS

The canonical candidate schema currently consists of decision metadata, candidate identity, split information, labels, and feature columns. Each row corresponds to exactly one candidate victim at exactly one full-cache miss decision. Trace identity and split metadata are recorded through `trace_name`, `trace_family`, `dataset_source`, and `split`. Cache configuration is recorded through `capacity` and `horizon`. Decision identifiers include `decision_id` and `decision_t`. The schema also records a per-decision chunk ID. That field is named `decision_chunk_id`. Candidate identity is represented through the field `candidate_page_id`. The labels are `y_loss`, the finite-horizon counterfactual miss cost after forcing that candidate eviction under the fixed continuation policy, and `y_value` = $-y_{\text{loss}}$. Feature columns cover request context, candidate context, predictor/LRU signals, cache summary statistics, and recent request or hit rates.

Beyond candidate rows, the release defines two derived views. The *decision view* aggregates all rows that share a decision identifier and summarizes each decision with candidate count, minimum loss, maximum value, the set of optimal candidate IDs, tie count, and regret summary statistics. The *pairwise view* compares candidate pairs from the same decision and derives pairwise preference labels from the ordering of their `y_loss` values, with ties when the losses are equal. The evaluated open release does not ship the full pairwise view; it ships a capped pairwise sample of 1,000,000 rows instead.

Operationally, the schema can be read as a layered projection over the same decision events: candidate rows are the atomic supervised objects, the decision view groups those rows by `decision_id`, and the shipped pairwise sample keeps only selected within-decision comparisons. Table 3 summarizes those column groups and derived fields directly in manuscript form.

Table 4: Release-package layout using release-relative paths for the evaluated benchmark snapshot.

artifact	relative_path
candidate rows	data/candidate_rows/
decision view	data/decision_view/decision_view.parquet
pairwise sample	data/pairwise_sample/pairwise_sample.parquet
release manifest	metadata/release_manifest.json
checksums	metadata/checksums.sha256

6 RELEASE CONSTRUCTION AND VALIDATION

The release pipeline is designed to package existing generated candidate rows into a benchmark artifact without requiring raw traces in the release package. The builder uses an out-of-core DuckDB path to ingest candidate shards, normalize split values, emit partitioned Parquet candidate rows, derive the decision view, optionally create a capped pairwise sample, and write release metadata and checksums.

The validation surface is part of the benchmark contribution. The validation workflow includes:

- schema completeness for candidate partitions,
- split normalization and split-set consistency,
- presence of selected families and absence of blocked families,
- duplicate candidate detection within decision groups,
- consistency of decision metadata within a decision,
- decision-view schema and count consistency,
- checksum verification.

The evaluated open-release snapshot passed metadata repair, the provided verification suite, publication-bundle validation, and end-to-end release validation. The validation evidence comes from manifest-backed counts, schema checks, split checks, duplicate detection, and checksum verification implemented in reproducible scripts. The manuscript therefore reports validation of the release package without relying on unpublished hosting details. It does not claim that final public hosting or DOI assignment is complete. It also does not treat superseded status notes as the authoritative source of scientific evidence.

Table 4 summarizes the release-package components used in the evaluation: candidate rows, derived benchmark views, manifest metadata, and checksums. Validation claims rely on those package contents together with reproducible validation scripts rather than on any standalone status note.

7 BENCHMARK TASKS AND BASELINES

LAFC-Evict supports four benchmark tasks from one release.

Candidate-level value prediction. Predict `y_loss` or `y_value` from a single candidate row. This is a pointwise task over the canonical benchmark unit and measures how well a model predicts finite-horizon counterfactual eviction quality for one candidate at one decision.

Best-candidate selection. Given all candidate rows for one decision, identify a member of the minimum-loss set. This task can be framed as ranking or decision-level classification and connects

most directly to the actual cache-eviction choice because it evaluates whether a method selects a benchmark-optimal victim within that decision.

Pairwise preference prediction. Predict whether candidate A is better than candidate B within the same decision, where “better” means lower y_{loss} or equivalently higher y_{value} . This task uses the pairwise view or the shipped pairwise sample and provides a simpler learning interface for within-decision preference modeling. Because equal losses induce ties, reports using this task need to state explicitly whether evaluation includes tie rows, excludes them, or maps them to a separate class. Within the shipped pairwise sample, the A/B assignment is randomized by a deterministic hash over stable decision and pair-identity fields (`orientation_method`: `deterministic_hash_v1`), not by candidate-identifier ordering, so the non-tie `label_a_better`/`label_b_better` split is balanced (about 49.8%/50.2%) rather than reflecting a construction-induced directional imbalance.

Candidate-label distribution characterization. Summarize the empirical label surface without training a predictive model. This characterization task studies distributions of y_{loss} , y_{value} , regret, tie frequency, and decision ambiguity across families, capacities, horizons, and splits. It is descriptive rather than competitive, but it is important for understanding benchmark difficulty and for interpreting downstream task results.

The empirical scope of this paper remains intentionally narrow. Rather than presenting a large model bake-off, we report lightweight baselines that can be executed directly from the committed release views and that establish reproducible reference points for later work.

The current results surface covers three such references. A linear regression baseline over candidate rows predicts y_{loss} from released features. It is intentionally weak: test MAE is 3.8739 with RMSE 4.7152 and $R^2 = 0.0026$, and the validation-split R^2 is slightly negative (-0.0506). Using that same linear score for within-decision selection nevertheless provides a reproducible decision-level floor. On the pairwise task, the earlier random and majority non-tie results remain useful sanity checks, but by themselves they do not test whether the released candidate features contain decision-relevant signal.

To probe that question without changing the released benchmark schema, we build an augmented feature view for evaluation only by joining released candidate-side features back onto the capped shipped pairwise sample. This produces a pairwise-sample-backed evaluation surface rather than a full-pairwise-universe-backed one, and no full quadratic pairwise materialization is performed. The shipped sample is itself tie-heavy: 878,262 of its 1,000,000 rows (87.8%) are exact ties, leaving 121,738 non-tie rows evaluated below. Ties remain common and are reported here as part of the label-surface characterization rather than discarded. Table 6 reports six non-tie pairwise baselines on that 121,738-row surface: the random and majority sanity checks, 1-D LRU-score and predictor-score models, a pairwise baseline induced by the released pointwise linear y_{loss} regressor, and a logistic regression over released feature differences.

Table 5: Pairwise-sample-backed summary statistics for the shipped sample view.

statistic	value
pairwise rows	1,000,000
split breakdown	test=102,024; train=737,896; val=160,080
family breakdown	wiki2018=250,872; cloudphysics=245,336; metakv=190,584; twemcache=169,064; metacdn=144,144
unique decisions represented	118,635
label distribution	a_better=60,673; b_better=61,065; tie=878,262
A/B orientation method	deterministic_hash_v1 (randomized, not lexicographic)

The resulting evidence is stronger than the earlier sanity checks alone. The logistic feature-difference model is the strongest pairwise baseline, reaching 0.9660 accuracy with 0.0824 log loss on the non-tie subset of the shipped sample. The linear-score and LRU-score baselines also outperform the random and majority checks, which indicates that the released candidate features retain decision-relevant signal for pairwise preference prediction. These numbers nevertheless warrant conservative interpretation. They improve the empirical benchmark evidence without changing the released candidate-row, decision-view, or shipped pairwise artifacts, and they do not support full-universe or online-policy claims.

The best-candidate baseline’s evaluator previously reported 2,366,413 decision groups against a manifest-backed decision-view count of 2,363,286. That mismatch traced to a decision-key bug: the evaluator grouped candidate rows by a 6-column key instead of the canonical 9-column decision key, which double-counted 3,127 decisions whose rows were not contiguous within a partition file. The corrected evaluator now groups by the canonical 9-column decision key and reports 2,363,286 decisions, an exact match with the decision view. A cheap decision-level floor computed directly from the decision view shows the dataset is heavily tied: a uniform-random selector’s expected optimal-selection rate is 0.9912 with expected regret 0.0088, both more favorable than the linear-score selector’s observed 0.8753 optimal-selection rate and 0.1251 mean regret purely because of tie density in the label surface, not because the linear score underperforms a stronger baseline; headline selection-rate and regret numbers should be read alongside this random floor rather than in isolation.

8 EMPIRICAL CHARACTERIZATION

The empirical characterization emphasizes benchmark structure rather than a large model bake-off. The release metadata establish the scale and heterogeneity of the benchmark, and the released candidate-row summaries add full label-distribution reporting without changing that focus.

Across 277,995,072 candidate rows, y_{loss} has mean 7.7561, standard deviation 4.6542, and 5/25/50/75/95th percentiles of 2/4/7/11/16. The corresponding y_{value} summary mirrors those values by sign, as expected from the benchmark definition. The resulting label surface is broad but still bounded by the finite horizon, which keeps both candidate-level regression errors and decision-level regret interpretable. Table 8’s exactly equal horizon counts (787,762 rows at

Table 6: Feature-based pairwise baselines on the non-tie subset (121,738 of 1,000,000 rows) of the shipped pairwise sample, evaluated after the A/B orientation fix (deterministic hash-based randomization, not lexicographic ordering). An evaluation-only augmented feature view joins released candidate features back to the capped sample. Results are pairwise-sample-backed rather than full-pairwise-universe-backed, and no full quadratic pairwise materialization is performed.

baseline	feature source / model type	accuracy	log loss
random non-tie	random-label sanity check	0.5006	0.6931
majority non-tie	empirical majority-class sanity check	0.5016	0.6931
LRU-score pairwise	1-D logistic on LRU-score difference	0.9225	0.2426
predictor-score pairwise	1-D logistic on predictor-score difference	0.5016	0.6931
linear-score pairwise	logistic on pointwise linear-score difference	0.9490	0.1161
logistic feature difference	logistic regression on released feature differences	0.9660	0.0824

Table 7: Benchmark task definitions and representative evaluation outputs.

Task	Inputs	Representative outputs
Value regression	candidate rows	MAE, RMSE, and optional rank correlation between predicted and observed values
Best-candidate selection	decision groups	Top-1 accuracy against the benchmark-optimal set and mean realized regret relative to the best available candidate
Pairwise preference	pairwise rows from one decision	Accuracy or ROC-AUC, with tie handling stated explicitly and non-tie-only reporting marked when used
Label characterization	candidate or decision aggregates	Histograms, quantiles, tie rates, regret summaries, and per-family/capacity/horizon breakdowns

each of horizons 4, 8, and 16) reflect the same underlying full-cache-miss decisions relabeled at each configured horizon, whereas its capacity counts correspond to genuinely different full-cache-miss events produced by replaying each trace at each cache size; pooled statistics in this paper therefore mix horizons and capacities and should be read as aggregate release summaries rather than per-configuration results.

The same result summaries also show that the simple linear-score selector often lands in the benchmark-optimal set even though its underlying value regression is weak. That combination is useful descriptive context for later benchmark users: the release is not trivial, but a lightweight heuristic already clears a reproducible lower bar. The evaluator’s grouped decision count now matches the manifest-backed decision-view count exactly (Table 10); an earlier decision-key bug that double-counted a small number of decisions has been fixed and reconciled. Because the dataset is heavily tied at the decision level, a uniform-random selector’s expected optimal-selection rate and regret are included alongside the linear-score selector’s observed metrics so that headline numbers are not read in isolation from tie density.

Table 8: Decision-view-backed benchmark composition by family, capacity, horizon, and split.

dimension	counts
trace_family	wiki2018=598,560; cloudphysics=576,219; metakv=454,053; twemcache=397,494; metacdn=336,960
capacity	32=613,314; 64=599,172; 128=582,678; 256=568,122
horizon	4=787,762; 8=787,762; 16=787,762
split	test=242,418; train=1,746,882; val=373,986

Table 9: Decision-view-backed candidate-count statistics. These values summarize the number of candidates per decision in the preserved release.

group	min	med	mean	max
overall	32	64	117.63	256
cloudphysics	32	64	118.35	256
metacdn	32	64	117.12	256
metakv	32	64	119.52	256
twemcache	32	64	111.52	256
wiki2018	32	64	119.85	256

Table 10: Candidate-label and best-candidate-selection summaries from committed result JSON files.

statistic	value
candidate rows summarized	277,995,072
y_loss mean / std	7.7561 / 4.6542
y_loss q05 / q25 / q50 / q75 / q95	2 / 4 / 7 / 11 / 16
y_value mean / std	-7.7561 / 4.6542
y_value q05 / q25 / q50 / q75 / q95	-16 / -11 / -7 / -4 / -2
manifest-backed decision rows	2,363,286
best-candidate evaluator decision count	2,363,286 (exact match)
best-candidate optimal-selection rate	0.8753
best-candidate regret mean / median / max	0.1251 / 0 / 3
random-selector expected optimal-selection rate	0.9912
random-selector expected regret	0.0088

9 LIMITATIONS, ETHICS, AND REUSE

This benchmark has several important constraints. First, the main label is a finite-horizon continuation-policy-specific counterfactual target. It is decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label for the full future control problem. Good performance on this label therefore does not automatically imply online deployment gains.

Second, the labels depend on the continuation policy used after the forced eviction. A different continuation policy or a different horizon can change the numerical target, the optimal set within a decision, and the induced pairwise preferences. Comparisons are therefore meaningful only across artifacts that keep these label-generation choices fixed; the shipped pairwise sample is likewise capped and tie-heavy (Section 7).

Third, the evaluated release uses unit miss cost and object-count capacities (32, 64, 128, or 256 objects); variable object sizes in the upstream traces are not modeled in this label family, so labels do not capture byte-weighted miss cost or size-aware eviction trade-offs.

Fourth, the open subset is intentionally narrower than the larger generated dataset mix. Two trace families remain excluded from the open subset for redistribution reasons, and not every selected family appears in every split in the evaluated release package. This release-governance boundary is part of the benchmark story and is made explicit rather than hidden in artifact packaging details.

Fifth, benchmark reuse must preserve provenance distinctions. The release exposes generated supervision and derived tasks, but it does not erase the attribution, citation, or licensing requirements of the upstream raw traces from which candidate rows were derived. Those raw traces remain external artifacts with their own governance constraints.

Sixth, the evaluated open-release snapshot is distinct from any later public distribution channel; the manuscript centers that validated release package and its reproducible validation evidence rather than final hosting details.

Finally, the generated labels are intended for offline benchmarking claims. They can support comparative modeling studies, but they are not automatic evidence that a learned policy is ready for operational deployment without separate online evaluation, systems integration, and safety review. The paper also does not report full-policy evaluations against external eviction algorithms such as SIEVE, S3-FIFO, MAT, or Parrot; those methods are relevant comparison points and future benchmark consumers, but they are not claimed as evaluated baselines here.

10 RELATED WORK

10.1 Trace Datasets, Workload Studies, and Benchmark Surfaces

Public traces and benchmark infrastructure provide the substrate for cache evaluation. The UMass Trace Repository and large-scale workload studies of key-value stores and Twemcache clusters demonstrate the value of realistic, heterogeneous request streams for systems research [1, 29, 33]. More broadly, benchmark artifacts such as YCSB, OLTP-Bench, and the Join Order Benchmark made comparison easier by publishing reusable workloads, interfaces, and evaluation surfaces rather than one-off experiments [6, 7, 16]. Existing trace libraries and simulators therefore supply essential raw material for cache studies, and some public artifacts even expose oracle-style features or procedural oracle policies. LAFC-Evict packages a different layer of supervision: for each full-cache miss decision and candidate victim, it releases finite-horizon counterfactual loss labels under a fixed continuation policy, together with decision-level and pairwise views.

Those distinctions matter because raw traces, workload studies, and simulator-oriented artifacts do not by themselves provide a reusable table of candidate-level finite-horizon counterfactual outcomes. LAFC-Evict is intended to complement that ecosystem rather than replace it. The artifact is useful precisely when a downstream user wants to study offline value regression, best-candidate selection, preference learning, or label-distribution analysis without first regenerating the same counterfactual rollouts from scratch.

10.2 Learned Eviction and Imitation Learning

Recent learned-caching papers likewise contribute online policies, policy-selection schemes, or learning frameworks rather than reusable supervised benchmarks. LeCaR learns between replacement experts online [30], and CACHEUS extends that practical line with adaptive expert selection for cache replacement [23]. Parrot uses imitation learning to approximate Belady-style replacement decisions from past accesses [18], while Hawkeye and Mockingjay show how Belady-derived signals can guide deployable replacement policies in hardware caches [12, 25]. MAT focuses on reducing the overhead of ML-guided eviction by using a heuristic filter around a learned module [32]. ALPS emphasizes lightweight learned replacement and the practical cost of learning overheads [26], while Feedforward Neural Networks for Caching is useful precisely because it questions whether heavier learned models are always justified [10]. HR-Cache extends the learned-caching line into edge delivery settings [28]. Across these works, the primary contribution is a policy or policy-learning mechanism. By contrast, LAFC-Evict releases a static benchmark surface with candidate-level finite-horizon counterfactual labels, decision-view summaries, and a shipped pairwise sample for offline value regression, best-candidate selection, and preference-learning tasks. To our knowledge, we are not aware of a public artifact in this line that releases reusable per-candidate finite-horizon counterfactual supervision at this scale.

10.3 Modern Eviction Policies

Classical cache-replacement work defines the policy landscape that any learned benchmark must eventually interface with. Belady’s look-ahead policy remains the canonical offline reference point [2]. Practical online policies then emphasize different signals: LIRS focuses on inter-reference recency [14], ARC adaptively balances recency and frequency [21], CLOCK-Pro revisits CLOCK-style replacement with stronger reuse sensitivity [13], and GreedyDual-Size accounts for cost and size [3]. TinyLFU and W-TinyLFU further show that admission and segmented cache management matter in modern software caches [8, 9]. More recent object-cache work shows that policy progress does not always come from heavier machinery: S3-FIFO argues that simple FIFO-based queueing can beat more elaborate baselines at scale [34], while SIEVE emphasizes turn-key deployability for web caches [35]. Those policies are strong online comparison targets for future end-to-end studies, but they are not evaluated here because LAFC-Evict is an offline benchmark paper and does not claim a new deployed eviction policy. The contribution here is reusable supervision that can be used to train or diagnose such policies, not an online replacement rule that supersedes them.

10.4 Learning-Augmented Caching and Predictive Advice

Another relevant line of work treats caching as an online problem augmented by predictions rather than as a supervised dataset. Competitive Caching with Machine Learned Advice and later learning-augmented caching results formalize how predictive advice can improve classical online algorithms while preserving robustness guarantees [20, 31]. Robust Learning-Augmented

Caching: An Experimental Study extends that line empirically, while Parsimonious Learning-Augmented Caching studies how much prediction access is really needed [5, 11]. This perspective is complementary to LAFC-Evict rather than directly competing with it. Learning-augmented caching papers typically study online algorithms equipped with advice, predictor queries, and robustness objectives. By contrast, LAFC-Evict releases finite-horizon, continuation-policy-dependent counterfactual labels for every candidate within a decision. The artifact can train or diagnose predictors and eviction-scoring models before they are embedded into an online algorithm, and it supports value regression, best-candidate selection, pairwise preference learning, and label characterization from one release.

10.5 Benchmark and Dataset Positioning

Public learning-to-rank collections such as LETOR and the Yahoo! Learning to Rank Challenge show how reusable labeled artifacts can organize a research community [4, 19, 22]. In that limited sense, LAFC-Evict is to learned cache eviction somewhat analogous to LETOR-style ranking corpora: a standardized candidate-level label surface reduces bespoke label generation and evaluation drift. The label design in LAFC-Evict also connects to counterfactual learning and ranking because each cache decision induces a candidate set, a minimum-loss subset, regret summaries, and a natural pairwise structure [15, 17, 24, 27]. Even so, LAFC-Evict is not simply a ranking dataset transplanted into systems. Its labels are tied to finite-horizon eviction outcomes under a specified continuation policy. The central differentiation to maintain throughout is therefore cautious but concrete: to our knowledge, we are not aware of a public cache-eviction artifact that releases reusable candidate-level finite-horizon counterfactual supervision together with decision-level and pairwise task views at this scale. In that sense, LAFC-Evict is neither a new online policy nor merely a raw trace repository or simulator-centric contribution, but a reusable benchmark artifact for learned cache-eviction research that complements existing traces, simulators, and policy papers.

11 CONCLUSION

This paper positions LAFC-Evict as a benchmark artifact for learned cache eviction rather than as a new policy paper. The core contribution is a release-oriented package built around candidate-level counterfactual supervision, derived decision and pairwise tasks, and validation-aware artifact construction. The preserved open release now couples manifest-backed scale and successful end-to-end validation with committed summaries of candidate-label distributions, a lightweight linear value-regression baseline, a derived best-candidate selector, and pairwise-sample-backed feature baselines that show measurable decision-relevant signal in the released candidate features.

Those results warrant conservative interpretation. The linear regression baseline is weak, the best-candidate selector is a reproducible benchmark floor rather than deployment evidence, the pairwise results remain backed by the shipped capped sample rather than any full quadratic pairwise universe, and the labels remain finite-horizon continuation-policy-dependent counterfactuals rather than globally optimal online targets. Future extensions

can focus on richer per-family and per-horizon characterization, stronger learned baselines, online policy studies that sit above the benchmark layer, and broader public hosting where redistribution constraints permit it.

USE OF GENERATIVE AI TOOLS

The authors used generative AI and AI-assisted coding and search tools to support manuscript drafting and editing, related-work exploration, code development, debugging, experiment organization, and result and table summarization. All scientific claims, related-work interpretations, code, experiments, tables, and final manuscript text were reviewed and verified by the authors, who remain fully responsible for the content and correctness of this work. No AI tool is listed as an author or as a contributor to authorship.

REFERENCES

- [1] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload Analysis of a Large-Scale Key-Value Store. In *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 53–64.
- [2] L. A. Belady. 1966. A Study of Replacement Algorithms for a Virtual-Storage Computer. *IBM Systems Journal* 5, 2 (1966), 78–101. <https://doi.org/10.1147/sj.52.0078>
- [3] Pei Cao and Sandy Irani. 1997. Cost-Aware WWW Proxy Caching Algorithms. In *Proceedings of the USENIX Symposium on Internet Technologies and Systems*. 193–206. <https://www.usenix.org/conference/usits-97/cost-aware-www-proxy-caching-algorithms>
- [4] Olivier Chapelle, Yi Chang, and Tie-Yan Liu (Eds.). 2011. *Proceedings of the Yahoo! Learning to Rank Challenge*. JMLR Workshop and Conference Proceedings, Vol. 14. JMLR.org. <https://proceedings.mlr.press/v14/>
- [5] Jakub Chledowski, Adam Polak, Bartosz Szabucki, and Konrad Tomasz Żołna. 2021. Robust Learning-Augmented Caching: An Experimental Study. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 1920–1930. <https://proceedings.mlr.press/v139/chledowski21a.html>
- [6] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the ACM Symposium on Cloud Computing*. 143–154.
- [7] Djelle Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proceedings of the VLDB Endowment* 7, 4 (2013), 277–288.
- [8] Gil Einziger, Ohad Eytan, Roy Friedman, and Ben Manes. 2018. Adaptive Software Cache Management. In *Proceedings of the 19th International Middleware Conference*. 94–106. <https://doi.org/10.1145/3274808.3274816>
- [9] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage* 13, 4 (2017), 35:1–35:31. <https://doi.org/10.1145/3149371>
- [10] Vladyslav Fedchenko, Giovanni Neglia, and Bruno Ribeiro. 2018. Feedforward Neural Networks for Caching: Enough or Too Much? *SIGMETRICS Performance Evaluation Review* 46, 3 (2018), 139–142. <https://doi.org/10.1145/3308897.3308958>
- [11] Sungjin Im, Ravi Kumar, Aditya Petety, and Manish Purohit. 2022. Parsimonious Learning-Augmented Caching. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 9588–9601. <https://proceedings.mlr.press/v162/im22a.html>
- [12] Akanksha Jain and Calvin Lin. 2016. Back to the Future: Leveraging Belady’s Algorithm for Improved Cache Replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture*. 78–89. <https://doi.org/10.1145/3007787.3001146>
- [13] Song Jiang, Feng Chen, and Xiaodong Zhang. 2005. CLOCK-Pro: An Effective Improvement of the CLOCK Replacement. In *Proceedings of the Annual USENIX Technical Conference*. 323–336. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>
- [14] Song Jiang and Xiaodong Zhang. 2002. LIRS: An Efficient Low Inter-Reference Recency Set Replacement Policy to Improve Buffer Cache Performance. In *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 31–42. <https://doi.org/10.1145/511334.511340>
- [15] Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased Learning-to-Rank with Biased Feedback. In *Proceedings of the 10th ACM International Conference on Web Search and Data Mining*. 781–789.

- [16] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *The VLDB Journal* 27, 5 (2018), 643–668. <https://doi.org/10.1007/s00778-017-0480-7>
- [17] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. 2011. Unbiased Off-line Evaluation of Contextual-Bandit-Based News Article Recommendation Algorithms. In *Proceedings of the 4th ACM International Conference on Web Search and Data Mining*. 297–306. <https://doi.org/10.1145/1935826.1935878>
- [18] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An Imitation Learning Approach for Cache Replacement. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 6237–6247. <https://proceedings.mlr.press/v119/liu20f.html>
- [19] Tie-Yan Liu. 2009. Learning to Rank for Information Retrieval. *Foundations and Trends in Information Retrieval* 3, 3 (2009), 225–331. <https://doi.org/10.1561/1500000016>
- [20] Thodoris Lykouris and Sergei Vassilvitskii. 2021. Competitive Caching with Machine Learned Advice. *J. ACM* 68, 4 (2021), 24:1–24:25. <https://doi.org/10.1145/3447579>
- [21] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. 115–130. <https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [22] Tao Qin, Tie-Yan Liu, Jun Xu, and Hang Li. 2010. LETOR: A Benchmark Collection for Research on Learning to Rank for Information Retrieval. *Information Retrieval* 13, 4 (2010), 346–374. <https://doi.org/10.1007/s10791-009-9123-y>
- [23] Liana V. Rodriguez, Farzana Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning Cache Replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 341–354. <https://www.usenix.org/conference/fast21/presentation/rodriguez>
- [24] Yuta Saito. 2020. Unbiased Pairwise Learning from Biased Implicit Feedback. In *Proceedings of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval*. 5–12. <https://doi.org/10.1145/3409256.3409812>
- [25] Ishan Shah, Akanksha Jain, and Calvin Lin. 2022. Effective Mimicry of Belady’s MIN Policy. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 558–572. <https://doi.org/10.1109/HPCA53966.2022.00048>
- [26] Rana Shahout and Roy Friedman. 2024. Sketching the Path to Efficiency: Lightweight Learned Cache Replacement. In *27th International Conference on Principles of Distributed Systems (OPDIS 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 286)*. 34:1–34:21. <https://doi.org/10.4230/LIPIcs.OPDIS.2023.34>
- [27] Adith Swaminathan and Thorsten Joachims. 2015. Counterfactual Risk Minimization: Learning from Logged Bandit Feedback. In *Proceedings of the 32nd International Conference on Machine Learning*. 814–823. <https://proceedings.mlr.press/v37/swaminathan15.html>
- [28] Hoda Torabi, Hamzeh Khazaei, and Marin Litoiu. 2024. A Learning-Based Caching Mechanism for Edge Content Delivery. In *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*. 236–246. <https://doi.org/10.1145/3629526.3645037>
- [29] University of Massachusetts Amherst. 2025. UMass Trace Repository. <https://traces.cs.umass.edu/> Official trace repository website.
- [30] Giuseppe Vietri, Liana V. Rodriguez, William A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-based LeCaR. In *Proceedings of the USENIX Workshop on Hot Topics in Storage Systems (HotStorage)*. <https://www.usenix.org/conference/hotstorage18/presentation/vietri>
- [31] Alexander Wei. 2020. Better and Simpler Learning-Augmented Online Caching. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 176)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 60:1–60:17. <https://doi.org/10.4230/LIPIcs.APPROX/RANDOM.2020.60>
- [32] Dongsheng Yang, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2023. A Learned Cache Eviction Framework with Minimal Overhead. <https://doi.org/10.48550/arXiv.2301.11886> arXiv:2301.11886 [cs.OS]
- [33] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2020. A Large-Scale Analysis of Hundreds of In-Memory Cache Clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 191–208. <https://www.usenix.org/conference/osdi20/presentation/yang>
- [34] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. 2023. FIFO Queues Are All You Need for Cache Eviction. In *ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP ’23)*. <https://doi.org/10.1145/3600006.3613147>
- [35] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 1229–1246. <https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo>